

Real-Time Sign Language Recognition System

Name: Yash Pratap Singh

Session: 2025–26

Subject Area: GenAI

1. The Problem

I realized that communicating with someone who uses sign language isn't easy if you don't understand it. I have to see if a laptop could know some how what someone is saying just by watching their hands. Most existing apps use raw pictures, which makes them slow. They also stop working if the room is too dark or if there is a lot of mess in the background. If the sun sets or you have a busy wallpaper, those pixel-heavy models usually fail. I built a light system that reads gestures from a basic webcam and turns them into text and speech. It works in a normal living room without needing perfect studio lighting.

Real-Time Sign Recognition System

2. Design and Process

I didn't use standard image networks. Instead, I ignored the actual pixels and looked at the geometry of the hand. I used Google MediaPipe to track 21 specific points for every frame. Each point has x, y, and z coordinates measured from the wrist. This gives the computer 63 numbers that describe the hand shape in 3D. By skipping the camera pixels, I kept the model size at 5 MB. Regular image models are often 50 to 200 MB because they process every color value in a frame. This method means the system ignores what you are wearing or your skin color. It only cares about the joints. The code runs much faster on cheap hardware because the processor isn't busy looking at background noise.

3. Tracking Movement

A sign is a movement, not a still photo. I sent the 63 coordinates into a Bidirectional LSTM network. This setup looks at 30 frames at once and reads the data both forward and backward. Reading it in two directions helps the computer understand signs where the main motion happens at the very end. For example, when signing "thanks," the final downward move is what actually defines the word. At the moment, the model handles 10 ASL signs in real time and it doesn't need a high-end GPU.

4. Data and Preprocessing

Since I couldn't find any dataset based on 3D hand landmarks, I ended up creating one myself. I recorded 30 sequences for 10 signs: hello, thanks, yes, no, stop, peace, ok, fist, call, and point. I picked these because they look different from each other and work well for basic needs. Each clip is about a second long. Before training, I used a scaler to adjust the coordinates so they have zero mean and unit variance. Because of this, the model focuses more on the hand shape itself rather than position. It doesn't matter if I stand far away or close to the lens. I saved this scaler to a file. I saved the scaler so the webcam input during the demo matches my training data. I used 80% of the data for training and kept 20% for testing and I also mixed some random noise into the training set. This keeps the model from just memorizing the patterns. It needs to learn how to work with the different things it will see in real life.

5. Live App and Interface

The live app runs in a loop. The webcam takes a picture, MediaPipe finds the 21 points, and these go into a buffer of 30 frames. Once the buffer is full, the model guesses the sign. If it is more than 75% sure, the word pops up on the screen. The computer then speaks the word out loud using a background process. I added three visual tools to help the user: Attention Heatmap: A color bar that shows which part of the movement the model liked best. Green areas usually show where the hand moved the most. Skeleton Mirror: A black window that draws the hand landmarks as dots and lines. This shows exactly what the computer sees. Confidence Meter: A bar that changes from orange to green. It gives the user instant feedback on whether they are signing clearly.

6. Results and Fixes

The model reached 95% accuracy on the test data. It worked well for most signs. I did find some mistakes between "hello" and "thanks" because both use an open palm near the face. I hit a few snags during coding. The live video used to freeze every time the computer spoke. This happened because the speech function was stopping the main video loop. I moved the speech task to a separate thread to keep the video smooth. I also found the model failed at first because I forgot to scale the live coordinates. The model wanted small numbers, but the camera was sending raw pixel values. Loading the saved scaler from the training phase fixed this.

7. What is Next

I want to track both hands in the next version. This will let the system understand harder signs like "family" or "work." I also want to add more common phrases. Since the model is small, I can turn it into an offline phone app. I want to talk to other people building tools for accessibility to see how they handle two-hand signs.

